The Build: how Pharmer reads the fine print

The technical part of our team's How-we-did-it report: the tools, the pipeline, the prompts that carry the rules, where the AI was confidently wrong, and what we would change.

In one paragraph

We did not run the analysis as a chat. We built **Pharmer**, a Python pipeline that takes a ticker, pulls the company's own SEC filings and FDA labels, has six specialist AI 'hunters' propose findings with page-cited verbatim quotes, and then checks, sizes and ranks those findings **in code** before a human picks the Sharpest Needle. Only after that pick do the AI stages write a verdict and a CEO move, and the tool renders a footnoted PDF. Each stage's code knows nothing about a particular company, so the same build runs on any of the ten shortlist companies. A full live run on **Eli Lilly** completed on Partner A's machine; its findings are in the deck. Two people built it in parallel over one weekend: 36 merged pull requests and 503 offline tests, all passing.

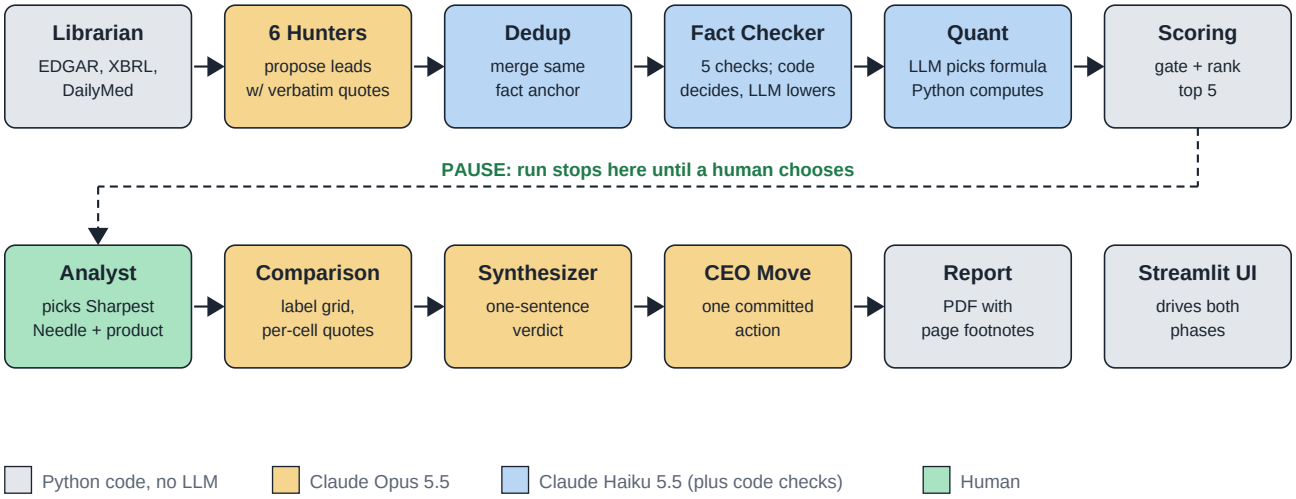

Figure 1. The Pharmer pipeline. Colour shows who does the work at each stage. The dashed line is the human checkpoint: the run stops after Scoring and waits for the analyst.

1

The tools we used, and what each was for

Tool	What it was for
Claude Code (Opus 5.5)	Our build environment. Each partner ran their own session. It wrote the code, tests and agent prompts, ran the tests, and drove headless Chrome to screenshot the UI. Humans made every design and scope decision.
Claude API: Opus 5.5	The judgement-heavy agents inside the pipeline: the six hunters, the Comparison Builder, the Synthesizer and the CEO Move.
Claude API: Haiku 5.5	Mechanical stages: the Dedup judge (are two leads the same story?), the Fact Checker's claim-support judge, and Quant's formula choice.
Codex (OpenAI)	An independent review of the codebase by a second model family, reading code that Claude wrote. It is less likely to share Claude's blind spots.
BMAD skills in Claude Code	Structured brainstorming for the pipeline design and the UI plan, and multi-reviewer code reviews (security, edge-case and adversarial lenses) on every major stage.
SEC EDGAR + XBRL companyfacts	Primary source: 10-K, 10-Q, 20-F and 6-K filings, plus structured figures used as a second, independent source for every number.
DailyMed (FDA labels)	Current prescribing information for the product and its rivals.
ECB reference rates	FX conversion for non-USD filers (Novo reports in DKK). We fetch the rates from the ECB and never take them from the model.
Python stack	asyncio orchestrator, reportlab (PDF), Streamlit (UI), unittest with a fake Claude client. Every API call is logged per request to `logs/api_usage.csv`.
GitHub	One branch and one PR per change. The repo docs (architecture, data contracts, build log) were the shared memory between two AI sessions.

2

Workflow

How documents got into the AI

- **No PDFs pasted into a chat.** The Librarian (pure Python, no LLM call) fetches filings from EDGAR and splits them into chunks. Each chunk carries a `chunk_id`, the document, the section and the filing's own *printed* page number. When a filing has no printed page number, the chunk is flagged as an inferred page.
- It selects the right document by content, not by filename. Novo's 4.4 MB remuneration report scored 1-2 on our financial markers. Its 2.8 MB Q2 financial report scored 17. Picking by size would have chosen the wrong file.
- Tables keep their cell text verbatim, for example `(1,234)` stays as printed, so a quote can later be string-matched against the source.
- Hunters never see the whole corpus in their prompt. They get four tools (`search_chunks`, `read_chunks`, `get_facts`, `submit_leads`), and `submit_leads` **rejects** any unknown chunk or any quote that is not a character-for-character substring of its chunk. The model has to fix its own citation before anything is recorded.
- FDA labels come from DailyMed and are accepted only when every package code belongs to the drug's maker. A repackager's copy of the Ozempic label was filed under the same application number, and that rule kept it out.

How work was split between people and tools

Who	What they owned
Partner A (data + checks)	Librarian, Fact Checker, Quant: getting the evidence in and deciding whether it holds.
Partner B (agents + output)	Hunters, Dedup, scoring, Comparison Builder, Synthesizer, CEO Move, PDF report, Streamlit UI.
Humans only	Which company. The Sharpest Needle. The product and its rivals. Scope cuts (e.g. deferring the Priced-In Detector and Peer Agent). Every UI option. Any change to the shared data contracts needed both partners.
AI (Claude Code)	Drafting code, tests and prompts against written specs, and first-pass review. Every decision it made that the code doesn't show was logged in `docs/build-log.md` for the humans to see.
AI (in the pipeline)	Reading and proposing (hunters), judging similarity and claim support (Haiku), and writing the verdict and the move (Opus). It never does arithmetic and never chooses the final finding.

How outputs were checked

Our rule was **"no fact from model memory."** So the checks live in code that the model cannot talk its way past:

Stage	Check enforced in code
Hunters	Quote must be verbatim in the cited chunk and must contain the anchor value. Otherwise the lead is bounced back to the model.
Fact Checker	Five checks: quote match, value carried by the quote, number-tie against XBRL, like-for-like period, claim support. The deterministic checks are authoritative. The Haiku judge can only *lower* confidence, never raise it.
Number-tie	Tolerance is 0.5×10^{-dp} of the printed decimals, not a percentage. 45,042.7 ties to 45,043. A 1% tolerance would have accepted 45,492, a different number.
Quant	The model names a formula and its inputs. Python evaluates it through an AST whitelist (+ - * / only). Each input must be a corpus fact, a verbatim quote, or a stated and justified assumption.
Synthesizer / CEO Move	Every number in the output must appear in that call's input. The CEO move must cite the Sharpest Needle and may not start with 'investigate', 'monitor', etc. On failure the model gets one retry, then the stage writes status *failed* and never invents.
Comparison grid	A cell is kept only if its quote is verbatim in a chunk sent to that column and every number is in the quote. Otherwise the cell shows "not disclosed".

During development we also checked the checkers. Every test runs offline against a fake Claude client. A network-blocked run of the suite proves zero calls. We hand-tied 24 XBRL facts to printed filing tables (Novo 8/8, Lilly 12/16 same-period, with the other 4 found in comparative columns), and we hand-sampled 14 citations back to the page they claim. Every major stage then went through a three-reviewer AI code review plus Codex, and every UI change was checked on a screenshot.

3

Our most important prompts

The prompts below are quoted verbatim from `pharmer/prompts/`. Each one is byte-identical on every call, so prompt caching reuses it. Each rule in a prompt is also enforced by code, because a prompt alone is a request, not a guarantee.

hunter_rules.md: shared by all six hunters

```
Use only the corpus you can reach with your tools. Never use facts, numbers, dates, quotes or
events from your own memory or training data, even if you are sure they are true.
...
a quote copied character-for-character from that chunk's text (same spacing, commas and line
breaks; keep quotes short, one line of a table is ideal).
```

Why it mattered: This is the case's warning ("a model working from memory will get numbers and quotes wrong") turned into an operating rule. The submit_leads tool rejects any quote that breaks it.

quant.md: sizing a finding

```
You size a verified finding. You do not compute the answer: Python evaluates your formula
against the inputs you name, so the number in the report is calculated and auditable rather
than generated.
...
Needing a conventional rate is not a reason to refuse.
```

Why it mattered: It splits judgement (which mechanism, which rate) from arithmetic. The second line was added after a live run where the model refused to size anything (see section 4).

fact_checker.md: the claim-support judge

```
A number can be perfectly correct and still not support the story told about it. That gap is
what you are here to find.
...
Your verdict can lower a finding's confidence; it can never raise it above what the
deterministic checks already allow.
```

Why it mattered: Code can confirm a number. Only a reader can tell whether "channel stuffing" follows from it. We gave the LLM that one narrow job and capped its power.

ceo_move.md: Part 2

```
It must be a decision, not more research: never "investigate", "monitor", "study", "review",
"explore", "assess", ... and never "we will investigate" or similar.
...
You must cite the Sharpest Needle.
```

Why it mattered: The case says AI will argue for almost any strategy. Without this rule the model's safe answer was more research. Code rejects these verbs as well.

Our standing instruction to Claude Code (CLAUDE.md, every session)

```
No fact from model memory: every number and quote comes from the corpus.
...
Append to docs/build-log.md (never edit old entries) whenever you: make a decision the code
doesn't show, catch the AI being confidently wrong, or finish a test.
```

Why it mattered: The build log is how two separate AI sessions stayed consistent. It is also the raw material for this report: every 'wrong' case below was logged when it happened.

4

Where the AI was confidently wrong, and how we caught it

Case A: a write-down that would have printed as a gain

What happened. On the first live Quant run the prompt asked in prose for "negative for damage". The model returned a **positive** magnitude for a write-down on every finding, with no hedging. In the report, a loss would have appeared as a gain.

How we caught it. By reading the first live run's output rather than trusting that the stage finished without errors.

Fix. The sign is no longer the model's choice. `direction` is a required `cost/benefit` field, and Python applies the sign. The same week, two live runs on *identical* Lilly evidence picked different write-down rates (one sized the two findings at -$36.7m / -$276.2m, the other at -$888.0m / -$462.4m), so the recommended finding **flipped**. Every figure was auditable, but the assumptions were not stable between runs. That is why a human, not the ranking, names the Sharpest Needle.

Case B: the same mistake, twice, through two different doors

First door. A web-search summary said Novo's Q2 sales grew **+7% at constant exchange rates**. The 6-K table showed reported **+3%**. Caught by opening the primary filing.

Second door. Days later, inside our own Fact Checker, the code matched table rows by prefix, so Novo's row `Net sales at CER` counted as `revenue`. That would have put a constant-currency figure into a reported-basis trend: the same confusion, coming back in. Caught by an independent reviewer. Labels now match by equality, with a regression test to keep it that way.

More from the build log (33 'wrong' entries in total)

What the AI asserted	Reality	How we caught it
Proposed openFDA as a backup label source.	It was missing Novo's own Ozempic label.	Compared against DailyMed.
Matched "Roche" to a company in the SEC list.	It was Sinopec Shanghai PETROCHEMICAL (a substring match).	Read the probe output.
Pfizer / J&J; prior-year quarters "missing".	They use 52/53-week fiscal quarters (Jun 28 vs Jun 29).	Listed the actual 10-Q dates.
A filing's page number is the last line of each page.	True for Lilly. Novo prints it in a running header, so every Novo page came out unverified.	Ran the second company instead of trusting the first.
The PDF said "Market view: not priced in".	No priced-in check had run. It was a default, not a finding.	Reviewing the rendered report.
A revenue trend from FY2007 to Q2 2026 in one series.	Mixed quarters with full years, so every Lilly finding was flagged "reversal".	Ran on the real Lilly corpus, not fixtures.
"No API key" handled gracefully, and the test passed.	The real path raised TypeError and killed the stage. The test injected a different error.	An edge-case reviewer ran it for real.
The offline test suite makes no network calls.	It was silently hitting the ECB live: 6.5599, not the fixture's 6.4407.	Two tests got the wrong FX rate.
The hunter prompt is neutral.	Its example (+25% receivables vs +3% sales) mirrored the real Novo story, priming the answer.	Three-reviewer code review.

The pattern. Almost nothing was caught by asking the AI whether it was sure. It was caught by going back to the primary document, by running a *second* real company, by a reviewer with a different lens or a different model (Codex), or by a test written to fail first. The checker needs checking too: one of our own verification scripts mis-paged three citations that were in fact correct.

5

What we would do differently

- **Prove the API works in the first hour.** Most stages were built and tested against a fake client before any real call succeeded. We lost hours to a truncated key (401) and then an organisation-scoped key that needed a workspace header (400). Today we would run our 5-check API smoke test before writing a single agent.
- **One corpus format from the start.** Partner B's sample corpus joined table cells with spaces and the Librarian used tabs. 132 chunk IDs collided and only 2 texts matched. We patched it in the Fact Checker. A shared fixture on day one would have prevented it.
- **Pin Quant's assumptions.** A reviewed table of write-down rates per metric would make the ranking stable across runs (Case A).
- **Get transcripts and peer filings in earlier.** They were the inputs for the Call Decoder, Peer Agent and Priced-In Detector, so those stages were cut. Without them, "not obvious" is not measured by the tool. The human Sharpest Needle pick is the safeguard.
- **Design the outputs from the case rubric backwards.** The CEO move file has no budget or rejected-alternatives fields, and Quant's native-currency figure has no contract field. The case asks for both. We found these gaps late, while building the UI.
- **Never change demo configuration late without a live check.** A last-minute PR that switched off unbuilt stages caused a regression after the push and was reverted.